



DBMS Notes

डेटाबेस मैनेजमेंट सिस्टम — पूरी गाइड

Beginner to Advanced | बिगिनर से एडवांस्ड तक



Colorful Notebook Style — highlighter definitions, sticky-note tips & diagrams

Table of Contents / विषय-सूची

1. परिचय (Introduction to DBMS)
2. डेटाबेस आर्किटेक्चर (Database Architecture)
3. डेटा मॉडल्स (Data Models)
4. ई-आर मॉडल (Entity-Relationship (ER) Model)
5. रिलेशनल मॉडल (Relational Model)
6. रिलेशनल अलजेब्रा (Relational Algebra)
7. एसक्यूएल (SQL) (Structured Query Language)
8. नॉर्मलाइज़ेशन (Normalization)
9. ट्रांज़ैक्शन (Transactions)
10. कंकरेंसी कंट्रोल (Concurrency Control)
11. रिकवरी सिस्टम (Recovery System)
12. इंडेक्सिंग और हैशिंग (Indexing & Hashing)
13. फाइल ऑर्गनाइज़ेशन (File Organization)
14. एडवांस्ड टॉपिक्स (Advanced Topics)
15. क्वेरी प्रोसेसिंग और ऑप्टिमाइज़ेशन (Query Processing & Optimization)
16. डेटाबेस डिज़ाइन — केस स्टडी (Database Design — Case Study)
18. एसक्यूएल प्रैक्टिस बैंक (SQL Practice Query Bank (30+ Queries))
19. हल किए गए समस्याएँ (Solved Conceptual Problems)
20. एमसीक्यू प्रैक्टिस बैंक (MCQ Practice Bank)
21. क्विक रिवीज़न (Quick Revision & Practice)

Introduction to DBMS

Data (डेटा) कच्चे facts और figures होते हैं — जैसे किसी student का नाम, roll number, marks । जब इस data को process करके किसी meaningful form में लाया जाता है, तो वो Information (सूचना) बन जाता है ।

📌 Database (डेटाबेस) — related data का एक organized collection होता है, जिसे इस तरह store किया जाता है कि उसे आसानी से access, manage और update किया जा सके ।

📌 DBMS (Database Management System) — एक software होता है जो database को create, maintain और control करने के लिए इस्तेमाल होता है । यह user और database के बीच एक interface की तरह काम करता है ।

► File System vs DBMS

शुरुआत में data को files में store किया जाता था (File-based System) । इसमें कई problems आती थीं, जिन्हें DBMS ने solve किया ।

Problem (File System)	DBMS Solution
Data Redundancy — same data बार-बार store होना	Centralized storage, redundancy कम
Data Inconsistency — अलग जगह अलग values	Single source of truth, consistency बनी रहती है
Difficult data access — नई query के लिए नया program	SQL से flexible querying
Data Isolation — data कई files में scattered	एक जगह integrated data
Concurrent access issues	Concurrency control built-in
Security problems	User-level access control
No backup/recovery	Automatic backup और recovery

► DBMS के Advantages (फ़ायदे)

- Redundancy Control — data की duplicate copies कम होती हैं
- Data Consistency — सभी जगह updated और सही data मिलता है
- Easy Data Access — SQL queries से आसान access
- Data Integrity — constraints की वजह से valid data ही store होता है
- Security — users को अलग-अलग permissions दी जा सकती हैं
- Backup and Recovery — crash होने पर data वापस मिल सकता है
- Concurrent Access — कई users एक साथ safely data access कर सकते हैं

📌 Exam Tip: DBMS के advantages याद रखने के लिए short-form बना लो — "RCEIS-BC" (Redundancy, Consistency, Easy access, Integrity, Security, Backup, Concurrency) /

► Applications of DBMS

- Banking — transactions, accounts, customer records
- Railway/Airline Reservation — booking, schedule management
- Universities — student records, admissions, results
- E-commerce — product catalog, orders, inventory
- Telecom — call records, billing
- Healthcare — patient records, prescriptions
- Social Media — posts, likes, connections, messages
- Government — Aadhar, voter ID, land records

► Data Abstraction (डेटा एब्सट्रैक्शन)

असली दुनिया के database बहुत complex होते हैं। DBMS users से इस complexity को छुपाकर सिर्फ ज़रूरी जानकारी दिखाता है — इसी को Data Abstraction कहते हैं। यह तीन levels पर काम करता है, जिसे हम अगले chapter (Database Architecture) में detail से पढ़ेंगे।

► Instance और Schema

📌 Schema — database का overall design/structure होता है (tables के नाम, columns, data types, constraints)। यह ज़्यादा बदलता नहीं है।

📌 Instance — किसी particular समय पर database में actually मौजूद data होता है। यह हर पल बदलता रहता है (insert/update/delete होने पर)।

📌 Schema को घर के नक्शे (blueprint) की तरह सोचो — वो fixed रहता है। Instance उस घर के अंदर मौजूद असली सामान की तरह है, जो बदलता रहता है।

► DBMS की Categories (Data Model के आधार पर)

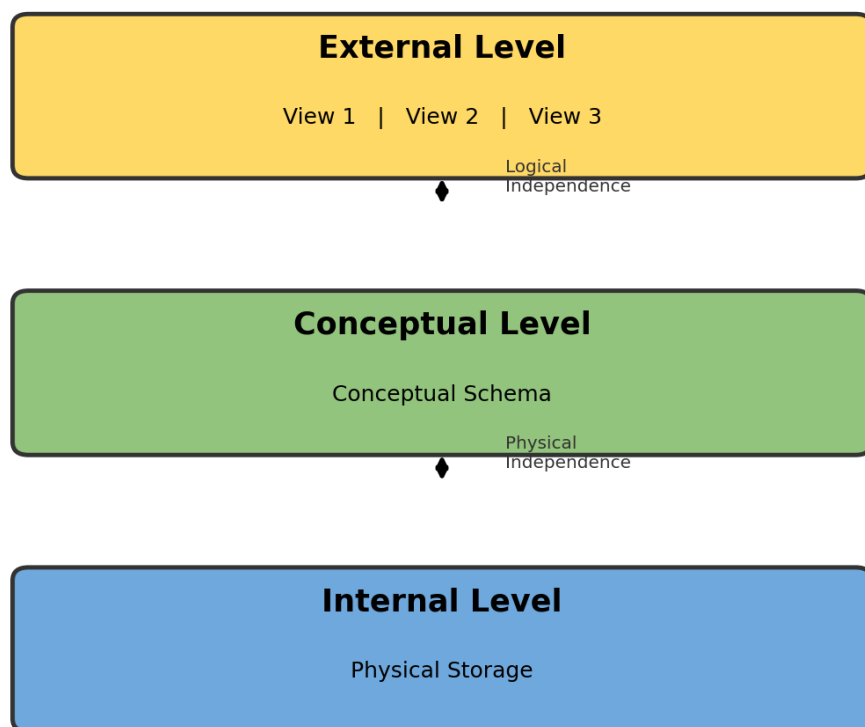
Category	Explanation
RDBMS (Relational)	Tables में data — MySQL, PostgreSQL, Oracle, SQL Server
OODBMS (Object-Oriented)	Objects के रूप में data — db4o, ObjectDB
Hierarchical DBMS	Tree structure — IBM IMS
Network DBMS	Graph structure — IDMS
NoSQL DBMS	Non-relational, flexible schema — MongoDB, Cassandra, Redis

► DBMS vs RDBMS

DBMS	RDBMS
Data files के रूप में store हो सकता है	Data हमेशा tables (relations) में store होता है
Relationships ज़रूरी नहीं	Tables के बीच relationships होते हैं (keys के through)
Normalization support नहीं करता	Normalization support करता है
उदाहरण: XML-based DBMS, File System DBMS	उदाहरण: MySQL, Oracle, PostgreSQL

DBMS की architecture ये बताती है कि data कैसे store होता है और users तक कैसे पहुँचता है । सबसे popular architecture है — Three-Schema Architecture (ANSI-SPARC) ।

Three-Schema Architecture



Three-Schema Architecture — External, Conceptual, Internal Levels

► तीन Levels

- 📌 **External Level (View Level)** — यह वो level है जहाँ अलग-अलग users को अपनी ज़रूरत के हिसाब से data का अलग-अलग view दिखता है । जैसे एक teacher को सिर्फ अपने students के marks दिखें ।
- 📌 **Conceptual Level (Logical Level)** — पूरे database का overall logical structure यहाँ describe होता है — कौन से tables हैं, उनके बीच क्या relationships हैं, क्या constraints हैं ।
- 📌 **Internal Level (Physical Level)** — यह बताता है कि data actually disk पर कैसे store है — files, indexes, storage structures ।

► Data Independence

Data Independence का मतलब है — एक level को change करने पर ऊपर वाले level पर असर नहीं पड़ना चाहिए ।

Type	Meaning
Logical Data Independence	Conceptual schema बदलने पर External schema (views) पर असर नहीं पड़े
Physical Data Independence	Internal schema (storage) बदलने पर Conceptual schema पर असर नहीं पड़े

 याद रखो: Logical independence achieve करना ज्यादा मुश्किल होता है क्योंकि applications सीधे logical structure पर depend करती हैं।

► DBMS के मुख्य Components

- Query Processor — SQL queries को parse और optimize करता है
- Storage Manager — data को disk पर read/write करता है
- Transaction Manager — ACID properties maintain करता है
- Buffer Manager — data को memory में temporarily hold करता है
- Metadata / Data Dictionary — database के बारे में जानकारी (schema, constraints)

► Users of DBMS

- Database Administrator (DBA) — पूरे database को manage करता है
- Application Programmers — software applications बनाते हैं जो DBMS use करते हैं
- Sophisticated Users — direct query language (SQL) से data access करते हैं
- Naive Users — simple forms/interfaces के through interact करते हैं
- Casual Users — कभी-कभार complex queries चलाने वाले users (जैसे management reports)

► DBA (Database Administrator) की Responsibilities

- Schema Definition — database का structure design करना
- Storage Structure और Access Method Definition
- Authorization — किसे कितनी access देनी है, तय करना
- Routine Maintenance — backup लेना, performance monitor करना
- Data की Integrity Constraints को Specify करना

► DBMS Component Diagram — काम कैसे होता है

जब कोई user कोई SQL query लिखता है, तो वह पहले DML Compiler के पास जाती है, फिर Query Optimizer उसे सबसे efficient तरीके से execute करने का plan बनाता है, इसके बाद Execution Engine, Storage Manager की मदद से actual disk से data fetch करता है।

Component	काम
DDL Interpreter	CREATE, ALTER जैसे DDL statements को process करता है
DML Compiler	SQL query को low-level instructions में translate करता है
Query Optimizer	query को execute करने का सबसे fast तरीका चुनता है
Execution Engine	optimized plan को actually run करता है
Storage Manager	disk से data read/write करने का काम करता है

 Query Optimizer एक ही query के कई possible execution plans में से सबसे कम cost वाला plan चुनता है— यह इंडेक्स, table size जैसी चीजों को ध्यान में रखता है।

Data Models

📌 Data Model — एक conceptual tool होता है जो बताता है कि data कैसे store, organize और manipulate किया जाएगा ।

► मुख्य Types

Model	विशेषता
Hierarchical Model	Data tree-structure में parent-child relationships के साथ organize होता है
Network Model	Records graph structure में जुड़े होते हैं, एक record के कई parents हो सकते हैं
Relational Model	Data tables (relations) के रूप में rows और columns में store होता है — आज सबसे popular
Object-Oriented Model	Data को objects के रूप में store किया जाता है (class, inheritance जैसे concepts)
ER Model	Real-world entities और उनके बीच relationships को design करने के लिए इस्तेमाल होता है

📌 Relational Model पर सबसे ज्यादा exam questions आते हैं — इसे अच्छे से समझो, आगे पूरा chapter इसी पर है ।

► Relational Model क्यों popular है?

- Simple structure — table के रूप में समझना आसान
- SQL जैसी powerful query language
- Mathematical foundation (Set Theory, Relational Algebra)
- Data Integrity को constraints से enforce किया जा सकता है
- Data Independence अच्छे से support होती है

► Hierarchical Model — थोड़ा detail में

इसमें data parent-child tree structure में organize होता है । हर child record का सिर्फ एक parent हो सकता है (one-to-many relationships) । उदाहरण: एक Organization chart, जहाँ एक Manager के नीचे कई Employees हों ।

फ़ायदे	नुकसान
Simple और fast — parent-child access तेज़ है	Many-to-Many relationships represent करना

फ़ायदे	नुकसान
	मुश्किल
Data की Integrity अच्छी रहती है	Structure बदलना (नया parent जोड़ना) मुश्किल है

► Network Model — थोड़ा detail में

यह hierarchical model का extension है, जिसमें एक child record के कई parents हो सकते हैं (graph-like structure) । इससे many-to-many relationships represent करना आसान हो जाता है, पर structure complex हो जाता है ।

► Object-Oriented Model

इसमें data को objects के रूप में store किया जाता है, जिनमें attributes (data) और methods (behavior) दोनों होते हैं — जैसे programming की OOP concepts (class, inheritance, encapsulation) । यह complex data types (जैसे images, videos) को handle करने में बेहतर है ।

► Object-Relational Model (ORDBMS)

🔗 Object-Relational Model — Relational Model और Object-Oriented Model का combination है । यह traditional tables के साथ-साथ object-oriented features (जैसे complex data types, inheritance) भी support करता है । उदाहरण: PostgreSQL ।

Entity-Relationship (ER) Model

ER Model का इस्तेमाल database design के शुरुआती stage में होता है — real world की चीज़ों को entities और उनके बीच relationships के रूप में represent करने के लिए ।

ER Diagram — Basic Symbols



ER Diagram के Basic Symbols

► मुख्य Terms

- 📌 Entity — कोई भी real-world object या concept जिसके बारे में data store करना है (जैसे Student, Course, Employee) ।
- 📌 Entity Set — एक ही type की सारी entities का collection (जैसे सभी Students का set) ।
- 📌 Attribute — entity की कोई property या characteristic (जैसे Student का name, roll_no) ।

► Attributes के Types

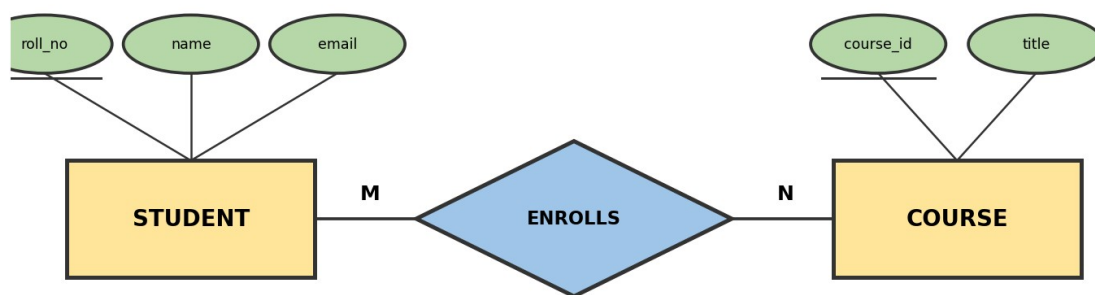
Type	उदाहरण / Explanation
Simple Attribute	आगे divide नहीं हो सकता — जैसे roll_no
Composite Attribute	छोटे parts में divide हो सकता है — जैसे Name = First + Last
Single-valued	एक ही value होती है — जैसे DOB
Multi-valued	एक से ज़्यादा values हो सकती हैं — जैसे Phone Numbers
Derived Attribute	किसी और attribute से calculate होता है — जैसे Age from DOB
Key Attribute	entity को uniquely identify करता है — जैसे roll_no

► Relationship और Cardinality

- ✎ Relationship — दो या ज़्यादा entities के बीच association को दिखाता है (जैसे Student 'enrolls in' Course) ।
- ✎ Cardinality Ratio — यह बताता है कि एक entity, दूसरी entity के कितने instances से relate हो सकती है ।

Cardinality	Example
One-to-One (1:1)	एक Person के पास एक Passport
One-to-Many (1:N)	एक Department में कई Employees
Many-to-One (N:1)	कई Students एक College में
Many-to-Many (M:N)	कई Students, कई Courses में enroll

Sample ER Diagram — Student enrolls in Course (M:N)



उदाहरण: Student — ENROLLS — Course (M:N Relationship)

► Weak Entity

- ✎ Weak Entity Set — वो entity जिसका अपना कोई key attribute नहीं होता, यह किसी strong (owner) entity पर depend करता है । इसे double rectangle से दिखाया जाता है । जैसे — Employee के Dependents ।

► Generalization, Specialization और Aggregation

- Generalization — कई entities की common properties को combine करके एक higher-level entity बनाना (bottom-up)
- Specialization — एक entity को उसकी sub-characteristics के आधार पर छोटे entities में divide करना (top-down)
- Aggregation — जब relationship को खुद entity की तरह treat करना पड़े ताकि उससे आगे relationship बनाई जा सके

📌 Generalization = Bottom-Up (छोटे से बड़ा), Specialization = Top-Down (बड़े से छोटा) — यह याद रखने का सबसे आसान तरीका है।

► Participation Constraints

Type	Explanation
Total Participation	Entity set की हर entity को relationship में हिस्सा लेना ज़रूरी है (double line से दिखाते हैं)
Partial Participation	कुछ entities relationship में हो सकती हैं, कुछ नहीं (single line से दिखाते हैं)

उदाहरण: हर Employee का किसी Department में होना ज़रूरी है (Total Participation), पर हर Employee का किसी Project पर काम करना ज़रूरी नहीं (Partial Participation)।

► Extended ER (EER) Features

📌 Union / Category — जब एक entity, कई अलग-अलग superclasses में से किसी एक का member हो सकती है (जैसे Account Holder — Person या Organization हो सकता है)।

Constraint	Explanation
Disjoint	Sub-classes आपस में overlap नहीं करतीं — एक entity सिर्फ एक subclass में हो सकती है
Overlapping	एक entity एक से ज़्यादा subclasses में हो सकती है
Total Specialization	हर superclass entity, किसी-न-किसी subclass में ज़रूर होनी चाहिए
Partial Specialization	superclass की entity किसी subclass में हो भी सकती है, ना भी हो

► ER Diagram बनाने के Steps (Practical Example)

मान लो हमें एक Hospital System के लिए ER Diagram बनाना है — जिसमें Doctor, Patient, और Appointment हैं।

- Step 1: Entities पहचानो — Doctor, Patient, Appointment
- Step 2: हर entity के attributes पहचानो — Doctor(doc_id, name, specialization), Patient(patient_id, name, age)
- Step 3: Relationships पहचानो — Doctor 'examines' Patient
- Step 4: Cardinality decide करो — एक Doctor कई Patients को examine कर सकता है, और एक Patient कई Doctors से मिल सकता है (M:N)
- Step 5: Diagram draw करो, keys underline करो

► ER Diagram को Relational Schema में Convert करना

- हर Strong Entity एक अलग Table बनती है
- हर Simple Attribute, table का एक Column बनता है
- Composite Attribute के sub-parts अलग columns बनते हैं
- Multi-valued Attribute के लिए अलग table बनानी पड़ती है
- 1:1 और 1:N Relationship में Foreign Key से काम चल जाता है
- M:N Relationship के लिए एक नई junction/bridge table बनानी पड़ती है
- Weak Entity की table में owner entity की Primary Key, Foreign Key की तरह आती है

रिलेशनल मॉडल

Relational Model

Relational Model में data को Tables (जिन्हें Relations कहते हैं) के रूप में store किया जाता है। हर table में Rows और Columns होते हैं।

Relational Term	सामान्य नाम	Explanation
Relation	Table	Rows-columns में organized data
Tuple	Row / Record	एक single entry / entity instance
Attribute	Column / Field	किसी property का नाम
Domain	Data Type / Value Set	attribute की valid values की range
Degree	Columns की संख्या	table में कितने attributes हैं
Cardinality	Rows की संख्या	table में कितनी tuples हैं

► Keys in DBMS (बहुत important topic)

- 🔑 Super Key — attributes का वह set जो table की हर tuple को uniquely identify कर सके (extra attributes के साथ भी)।
- 🔑 Candidate Key — वह minimal Super Key जिसमें कोई भी unnecessary attribute ना हो।
- 🔑 Primary Key — Candidate Keys में से चुनी गई एक key, जिसे table की main unique identifier के रूप में इस्तेमाल किया जाता है। यह कभी NULL नहीं हो सकती।
- 🔑 Alternate Key — बाकी बचे हुए Candidate Keys जो Primary Key के रूप में नहीं चुने गए।
- 🔑 Foreign Key — एक table का attribute जो दूसरी table की Primary Key को reference करता है। यह दो tables के बीच relationship बनाता है।
- 🔑 Composite Key — जब एक से ज़्यादा attributes मिलकर एक tuple को uniquely identify करते हैं।

📌 याद रखो: हर Primary Key एक Candidate Key होती है, लेकिन हर Candidate Key Primary Key नहीं होती।

► Integrity Constraints

Constraint	Rule
Domain Constraint	हर attribute की value उसके defined domain में ही होनी चाहिए

Constraint	Rule
Entity Integrity Constraint	Primary Key कभी भी NULL नहीं हो सकती
Referential Integrity Constraint	Foreign Key की value या तो NULL हो या parent table की existing Primary Key से match करे
Key Constraint	हर tuple की Primary Key unique होनी चाहिए

► Solved Example — Keys पहचानना

मान लो एक table है: Student(roll_no, aadhar_no, email, name, dept_id) । यहाँ roll_no, aadhar_no और email — तीनों individually हर student को uniquely identify कर सकते हैं ।

Attribute Set	Key Type
{roll_no}, {aadhar_no}, {email}	हर एक अपने आप में Candidate Key है
{roll_no} (चुनी गई)	Primary Key
{aadhar_no}, {email}	Alternate Keys
{roll_no, name}	Super Key (पर minimal नहीं, इसलिए Candidate Key नहीं)
{dept_id} — Department table की Primary Key को reference करता है	Foreign Key

► Relational Model की Properties

- हर column में सिर्फ एक ही तरह का data होता है (atomic values)
- हर row unique होती है
- columns का कोई fixed order ज़रूरी नहीं (theoretically)
- rows का भी कोई fixed order ज़रूरी नहीं
- हर column का नाम table में unique होता है

रिलेशनल अलजेब्रा

Relational Algebra

🔑 Relational Algebra एक procedural query language है — यह बताती है कि result निकालने के लिए किस order में operations perform करने हैं।

► Basic Operations

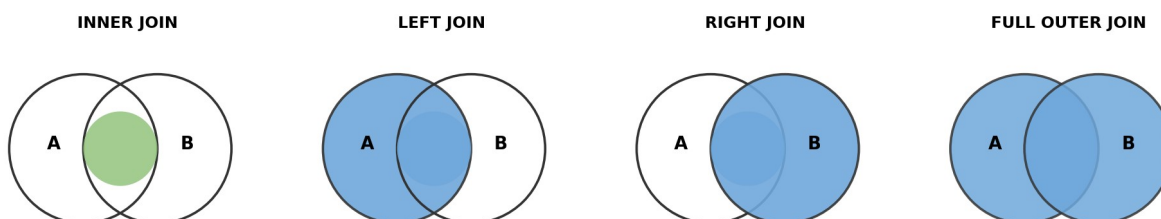
Operation	Symbol	Explanation
Select	σ (sigma)	Rows को filter करता है (condition के आधार पर)
Project	π (pi)	चुनी हुई Columns को निकालता है
Union	$\cup$	दो relations के tuples को combine करता है (duplicates हटाकर)
Set Difference	$-$	पहले relation में मौजूद पर दूसरे में ना हो, वो tuples
Cartesian Product	$\times$	दो relations के सभी combinations बनाता है
Rename	ρ (rho)	Relation या attribute का नाम बदलता है

उदाहरण: σ (marks > 80) (Student) — इसका मतलब है Student table से वो सारी rows निकालो जहाँ marks 80 से ज्यादा हैं।

► Derived / Extended Operations

- Intersection ($\cap$) — दोनों relations में common tuples
- Join ($\bowtie$) — दो relations को किसी common attribute के आधार पर combine करना
- Division ($\div$) — जब एक relation दूसरे relation की सभी values से associated हो

► SQL Joins के Types



Join Type	Explanation
INNER JOIN	सिर्फ वो rows जो दोनों tables में match करती हैं
LEFT (OUTER) JOIN	Left table की सभी rows + matching right rows (ना मिले तो NULL)
RIGHT (OUTER) JOIN	Right table की सभी rows + matching left rows (ना मिले तो NULL)
FULL OUTER JOIN	दोनों tables की सभी rows, match हो या ना हो
SELF JOIN	एक table को खुद के साथ join करना
CROSS JOIN	दोनों tables का हर combination (Cartesian Product)

► Relational Algebra — Solved Examples

```
-- Table: Employee(emp_id, name, salary, dept_id)

-- 1. Employees with salary above 50000
σ (salary > 50000) (Employee)

-- 2. Sirf naam aur salary columns nikalo
π (name, salary) (Employee)

-- 3. Select + Project combine (name of high earners)
π (name) (σ (salary > 50000) (Employee))

-- 4. Employee aur Department ko join karna
Employee ⋈ Employee.dept_id = Department.dept_id Department
```

► Relational Calculus (Brief Intro)

🔑 Relational Calculus — Relational Algebra के उलट, यह एक Non-Procedural (Declarative) language है — यह बताता है कि 'क्या' चाहिए, ना कि 'कैसे' निकालना है।

Type	Explanation
Tuple Relational Calculus (TRC)	Variables tuples को represent करते हैं
Domain Relational Calculus (DRC)	Variables domain की values को represent करते हैं

 SQL असल में Relational Calculus (declarative) पर based है, जबकि Relational Algebra internally query optimizer इस्तेमाल करता है।

एसक्यूएल (SQL)

Structured Query Language

🔑 SQL (Structured Query Language) — relational databases के साथ काम करने के लिए standard language है। इससे data को create, read, update, delete किया जाता है।

► SQL Commands की Categories

Category	Full Form	Commands	काम
DDL	Data Definition Language	CREATE, ALTER, DROP, TRUNCATE	Database structure define करना
DML	Data Manipulation Language	INSERT, UPDATE, DELETE	Data के साथ काम करना
DQL	Data Query Language	SELECT	Data को retrieve करना
DCL	Data Control Language	GRANT, REVOKE	Access control / permissions
TCL	Transaction Control Language	COMMIT, ROLLBACK, SAVEPOINT	Transactions को manage करना

► DDL Commands

```
-- Create a new table
CREATE TABLE Student (
    roll_no INT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    dept_id INT,
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
);

-- Modify a table
ALTER TABLE Student ADD email VARCHAR(100);

-- Delete a table completely
DROP TABLE Student;
```

► DML Commands

```
-- Insert data
```

```

INSERT INTO Student (roll_no, name, dept_id)
VALUES (1, 'Sushil', 101);

-- Update data
UPDATE Student SET name = 'Sushil Kumar' WHERE roll_no = 1;

-- Delete data
DELETE FROM Student WHERE roll_no = 1;

```

► SELECT — सबसे ज़्यादा इस्तेमाल होने वाला Command

```

-- Basic select
SELECT name, dept_id FROM Student;

-- With condition
SELECT * FROM Student WHERE dept_id = 101;

-- Sorting results
SELECT * FROM Student ORDER BY name ASC;

-- Grouping with aggregate function
SELECT dept_id, COUNT(*) AS total_students
FROM Student
GROUP BY dept_id
HAVING COUNT(*) > 5;

```

🔴 WHERE clause GROUP BY से पहले rows को filter करता है, जबकि HAVING clause grouped results को filter करता है / यह common exam question है /

► Aggregate Functions

Function	काम
COUNT()	Rows की गिनती करता है
SUM()	Numeric column का total निकालता है
AVG()	Average निकालता है
MIN()	सबसे छोटी value
MAX()	सबसे बड़ी value

► Joins in SQL

```
SELECT s.name, c.course_name
FROM Student s
INNER JOIN Enrollment e ON s.roll_no = e.roll_no
INNER JOIN Course c ON e.course_id = c.course_id;
```

► Subqueries

🔑 Subquery — एक query के अंदर लिखी गई दूसरी query, जिसका result बाहर वाली query use करती है।

```
SELECT name FROM Student
WHERE dept_id IN (SELECT dept_id FROM Department WHERE location = 'Delhi');
```

► Views

🔑 View — एक virtual table होती है जो किसी query के result पर based होती है। इसमें actual data store नहीं होता, हर बार query execute होती है।

```
CREATE VIEW HighScorers AS
SELECT name, marks FROM Student WHERE marks > 80;
```

► Constraints

Constraint	काम
PRIMARY KEY	Row को uniquely identify करता है, NULL नहीं हो सकता
FOREIGN KEY	दूसरी table से relationship बनाता है
NOT NULL	Column empty नहीं हो सकता
UNIQUE	सभी values अलग-अलग होनी चाहिए
CHECK	एक condition enforce करता है
DEFAULT	अगर value ना दी जाए तो default value use होती है

► Set Operations in SQL

```
-- UNION: दोनों results combine (duplicates हटाकर)
SELECT name FROM Student
UNION
SELECT name FROM Teacher;
```

```
-- UNION ALL: duplicates के साथ combine
SELECT name FROM Student
UNION ALL
SELECT name FROM Teacher;

-- INTERSECT: दोनों में common rows
SELECT roll_no FROM SportsTeam
INTERSECT
SELECT roll_no FROM MusicClub;

-- EXCEPT / MINUS: पहले में हो, दूसरे में ना हो
SELECT roll_no FROM Student
EXCEPT
SELECT roll_no FROM Graduated;
```

► String, Date और Numeric Functions

Function	काम
UPPER() / LOWER()	Text को uppercase/lowercase में बदलता है
LENGTH() / LEN()	String की length बताता है
SUBSTRING()	String का एक हिस्सा निकालता है
CONCAT()	दो या ज़्यादा strings जोड़ता है
NOW() / CURRENT_DATE	आज की date/time देता है
ROUND()	Number को round करता है
DATEDIFF()	दो dates के बीच का अंतर निकालता है

► Types of Subqueries

Type	Explanation
Single-row Subquery	सिर्फ एक row return करता है — =, <, > जैसे operators use होते हैं
Multi-row Subquery	कई rows return करता है — IN, ANY, ALL use होते हैं
Correlated Subquery	Outer query की value पर depend करता है, हर row के लिए re-evaluate होता है
Nested Subquery	Subquery के अंदर एक और subquery

```
-- Correlated Subquery example:
-- Har department mein sabse zyada salary wale employees
SELECT name, salary, dept_id
FROM Employee e1
WHERE salary = (
    SELECT MAX(salary) FROM Employee e2
    WHERE e1.dept_id = e2.dept_id
);
```

► Stored Procedures और Triggers — एक झलक

- ✍ Stored Procedure — SQL statements का एक group होता है जिसे एक साथ save करके बार-बार call किया जा सकता है, जैसे कोई function ।
- ✍ Trigger — एक special procedure है जो किसी table पर automatically execute होती है जब कोई specific event (INSERT, UPDATE, DELETE) होता है ।

```
-- Simple Trigger example
CREATE TRIGGER before_student_insert
BEFORE INSERT ON Student
FOR EACH ROW
BEGIN
    SET NEW.created_at = NOW();
END;
```

🔗 Triggers का ज्यादा इस्तेमाल business logic को database level पर enforce करने के लिए होता है — जैसे audit logging, automatic timestamps /

► SQL Query Order of Execution

SQL में लिखने का order और actual execution का order अलग होता है — यह समझना बहुत ज़रूरी है, exams में भी पूछा जाता है ।

Execution Order	Clause
1	FROM (और JOIN)
2	WHERE
3	GROUP BY
4	HAVING
5	SELECT

Execution Order	Clause
6	ORDER BY
7	LIMIT

► Solved Practice Queries

```
-- Table: Employee(emp_id, name, salary, dept_id, join_date)

-- Q1: Har department ka average salary, sirf 3+ employees wale
SELECT dept_id, AVG(salary) AS avg_sal
FROM Employee
GROUP BY dept_id
HAVING COUNT(*) >= 3;

-- Q2: Doosri sabse zyada salary find karna
SELECT MAX(salary) FROM Employee
WHERE salary < (SELECT MAX(salary) FROM Employee);

-- Q3: 2023 mein join hue employees
SELECT name FROM Employee
WHERE YEAR(join_date) = 2023;
```

नॉर्मलाइज़ेशन

Normalization

✍ Normalization — एक database design technique है जिससे data redundancy को कम किया जाता है और data की consistency बनाए रखी जाती है, tables को छोटे-छोटे related tables में तोड़कर ।

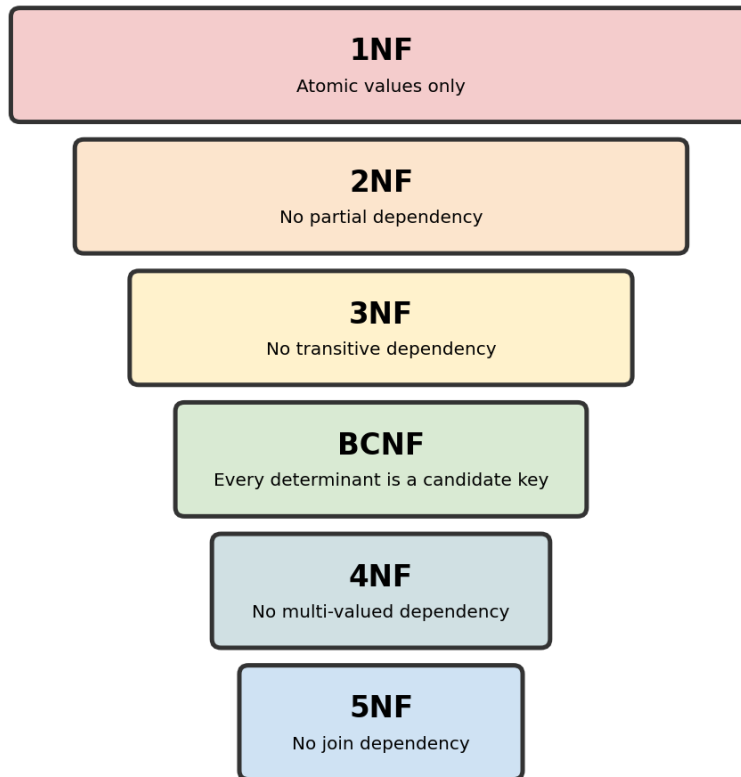
► Anomalies (बिना Normalization के होने वाली problems)

Anomaly	Explanation
Insertion Anomaly	नया data insert करने में दिक्कत, जब कुछ ज़रूरी information missing हो
Update Anomaly	एक ही data को कई जगह update करना पड़ता है, जिससे inconsistency हो सकती है
Deletion Anomaly	एक row delete करने पर ज़रूरी data भी गलती से चला जाना

► Functional Dependency

✍ Functional Dependency ($X \rightarrow Y$) — इसका मतलब है attribute X की value, attribute Y की value को determine करती है । यानी अगर दो rows में X की value same है, तो Y की value भी same होनी चाहिए ।

Normalization — Step by Step



Normal Forms — Step by Step (1NF से 5NF तक)

► Normal Forms

- 📌 1NF (First Normal Form) — हर column में सिर्फ atomic (indivisible) values होनी चाहिए, कोई repeating groups नहीं ।
- 📌 2NF (Second Normal Form) — table 1NF में हो और कोई भी non-key attribute, Primary Key के किसी part पर partially dependent ना हो (सिर्फ composite key वाले tables पर लागू) ।
- 📌 3NF (Third Normal Form) — table 2NF में हो और कोई transitive dependency ना हो (यानी non-key attribute किसी दूसरे non-key attribute पर depend ना करे) ।
- 📌 BCNF (Boyce-Codd Normal Form) — 3NF की एक stronger version । हर functional dependency $X \rightarrow Y$ में X जरूर एक Super Key होनी चाहिए ।
- 📌 4NF — कोई Multi-valued Dependency ना हो ।
- 📌 5NF — कोई Join Dependency ना हो, table को बिना data loss के आगे decompose नहीं किया जा सकता ।

📌 याद रखने का तरीका: 1NF = Atomic, 2NF = No Partial Dependency, 3NF = No Transitive Dependency, BCNF = Every determinant is a key.

► Decomposition

- Lossless Decomposition — tables को तोड़ने के बाद, उन्हें वापस join करने पर original data बिना loss के मिलना चाहिए
- Dependency Preserving Decomposition — decomposition के बाद, सभी original functional dependencies बरकरार रहनी चाहिए

► उदाहरण — Normalization Step by Step

मान लो एक table है जिसमें Student_ID, Student_Name, Course_ID, Course_Name, Instructor है। अगर एक student कई courses लेता है, तो Course_Name और Instructor बार-बार repeat होंगे — यह redundancy है।

```
-- Un-normalized Table
Student_ID | Student_Name | Course_ID | Course_Name | Instructor

-- After Normalization (2 separate tables)
Students(Student_ID, Student_Name)
Courses(Course_ID, Course_Name, Instructor)
Enrollment(Student_ID, Course_ID)  -- junction table
```

► पूरा Worked Example — 1NF से 3NF तक

मान लो हमारे पास एक table है: Orders(Order_ID, Customer_Name, Product_ID1, Product_ID2, Customer_City)। यहाँ एक order में 2 products के लिए अलग-अलग columns हैं — यह 1NF violate करता है (repeating group)।

Step	क्या करें
1NF में लाना	Product_ID1, Product_ID2 की जगह हर product के लिए अलग row बनाओ — Orders(Order_ID, Customer_Name, Product_ID, Customer_City)
2NF Check	अगर Primary Key = (Order_ID, Product_ID) है, तो Customer_Name सिर्फ Order_ID पर depend करता है (partial dependency) — इसे अलग table में निकालो
2NF Result	Orders(Order_ID, Customer_Name, Customer_City) + OrderItems(Order_ID, Product_ID)
3NF Check	Orders table में Customer_City, Customer_Name पर transitively depend कर सकता है — इसे भी अलग करो
3NF Result	Customers(Customer_Name, Customer_City) + Orders(Order_ID, Customer_Name) + OrderItems(Order_ID, Product_ID)

📌 हर normal form पहले वाले normal form की condition को भी satisfy करता है — यानी अगर table 3NF में है, तो वो automatically 1NF और 2NF में भी है।

► BCNF का Solved Example

मान लो table है: Enrollment(Student_ID, Course_ID, Instructor) | यहाँ मान लो हर Instructor सिर्फ एक Course पढ़ाता है (Instructor $\rightarrow$ Course), पर एक Student कई courses ले सकता है।

Candidate Key: (Student_ID, Course_ID) | पर Instructor $\rightarrow$ Course एक ऐसी dependency है जहाँ Instructor खुद Candidate Key नहीं है — यह BCNF violate करता है। इसे तोड़ना होगा: StudentInstructor(Student_ID, Instructor) + InstructorCourse(Instructor, Course_ID)।

► Multivalued Dependency (4NF)

📌 Multivalued Dependency ($X \twoheadrightarrow Y$) — जब attribute X, attribute Y की values के एक independent set को determine करता है, चाहे किसी और attribute की value कुछ भी हो।

उदाहरण: एक Student कई Languages जानता है और कई Hobbies रखता है, और दोनों independent हैं — इससे data redundant होने लगता है, इसलिए Languages और Hobbies को अलग-अलग tables में रखा जाता है (4NF)।

ट्रांजैक्शन

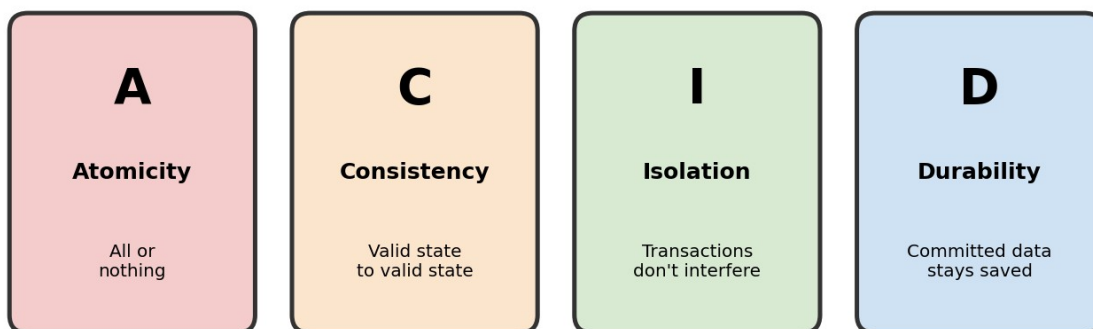
Transactions

✍ Transaction — database पर की गई operations (जैसे read, write) का एक logical unit होता है, जो या तो पूरी तरह complete होता है या बिल्कुल नहीं (all-or-nothing) ।

उदाहरण: bank transfer में — Account A से पैसे कटना और Account B में पैसे जुड़ना — यह दोनों operations मिलकर एक transaction बनाते हैं ।

► ACID Properties

ACID Properties of a Transaction

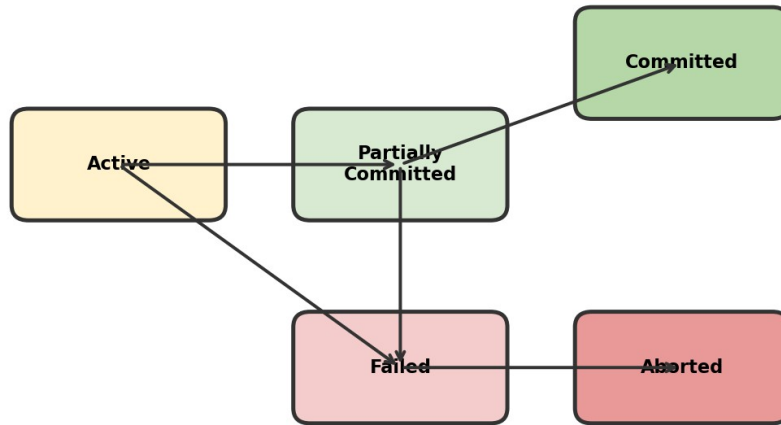


ACID Properties of a Transaction

Property	Explanation
Atomicity	Transaction या तो पूरी तरह execute हो, या बिल्कुल भी ना हो — बीच में रुकना allowed नहीं
Consistency	Transaction के पहले और बाद में database एक valid state में रहना चाहिए
Isolation	एक साथ चल रही transactions एक-दूसरे को affect नहीं करनी चाहिए
Durability	एक बार transaction commit हो जाए, तो उसका असर permanent रहता है — भले ही system crash हो जाए

► Transaction States

Transaction State Diagram



Transaction State Diagram

- Active — transaction execute हो रही है
- Partially Committed — सभी operations complete, पर changes अभी permanent नहीं हुए
- Committed — transaction successfully complete, changes permanent हो गए
- Failed — किसी error की वजह से आगे execute नहीं हो पा रही
- Aborted — transaction rollback हो गई, database पुरानी state में वापस आ गया

► Schedules

📌 Schedule — कई transactions के operations को execute करने का एक क्रम (order) होता है ।

Schedule Type	Explanation
Serial Schedule	Transactions एक के बाद एक (बिना interleaving के) execute होती हैं
Concurrent (Non-Serial) Schedule	अलग-अलग transactions के operations आपस में interleave होते हैं
Serializable Schedule	Concurrent schedule जिसका result किसी serial schedule जैसा ही हो — यह ideal होता है

► Serializability के Types

- Conflict Serializability — conflicting operations को swap करके अगर schedule को serial schedule में बदला जा सके
- View Serializability — अगर दोनों schedules का final result और read/write pattern same हो

📌 Conflicting operations वो होते हैं जो: (1) अलग-अलग transactions के हों, (2) same data item पर हों, और (3) कम से कम एक write operation हो ।

► SQL में Transaction Control

```
BEGIN TRANSACTION;

UPDATE Account SET balance = balance - 5000 WHERE acc_no = 'A101';
UPDATE Account SET balance = balance + 5000 WHERE acc_no = 'B202';

-- Agar dono updates safal, to commit karo
COMMIT;

-- Agar kisi step mein error aaye, to rollback karo
-- ROLLBACK;
```

🔑 **SAVEPOINT** — transaction के अंदर एक checkpoint बनाता है, जहाँ तक बाद में selectively rollback किया जा सकता है, पूरी transaction rollback किए बिना ।

► Solved Example — Serializability Check

मान लो दो transactions हैं: T1 (Read A, Write A) और T2 (Read A, Write A) । अगर schedule है: R1(A), R2(A), W1(A), W2(A) — यहाँ W1(A) और R2(A) conflict नहीं करते क्योंकि R2, W1 से पहले हुआ । पर W1(A) और W2(A) conflict करते हैं (दोनों writes हैं) । इस schedule में R1-W2 और R2-W1 के बीच conflicts check करके पता लगाया जाता है कि यह serializable है या नहीं ।

🔗 **Serializability check करने के लिए Precedence Graph (Serialization Graph) बनाओ** — अगर graph में कोई cycle नहीं है, तो schedule Conflict Serializable है ।

कंकरेंसी कंट्रोल

Concurrency Control

🔑 Concurrency Control — techniques का वह set है जो multiple transactions को एक साथ safely execute होने देता है, बिना data की consistency को नुकसान पहुँचाए।

► Concurrency ना Control होने पर होने वाली Problems

Problem	Explanation
Dirty Read	एक transaction, दूसरी uncommitted transaction का data पढ़ लेती है
Lost Update	दो transactions एक ही data update करती हैं, और एक update खो जाता है
Unrepeatable Read	एक transaction में same data को दो बार पढ़ने पर अलग values मिलती हैं
Phantom Read	एक transaction दोबारा query करने पर नई rows दिखने लगती हैं

► Lock-Based Protocols

🔑 Lock — एक mechanism है जो data item को access करने के अधिकार को control करता है, ताकि एक समय में सिर्फ authorized transaction ही उसे modify कर सके।

Lock Type	Explanation
Shared Lock (S)	सिर्फ read के लिए — कई transactions एक साथ shared lock ले सकती हैं
Exclusive Lock (X)	read और write दोनों के लिए — एक समय में सिर्फ एक transaction को मिलता है

🔑 Two-Phase Locking (2PL) — एक protocol है जिसमें transaction के दो phases होते हैं: Growing Phase (जिसमें सिर्फ locks लिए जाते हैं) और Shrinking Phase (जिसमें सिर्फ locks release किए जाते हैं)। एक बार लॉक release करने के बाद नया lock नहीं लिया जा सकता।

🔑 2PL serializability को guarantee करता है, लेकिन deadlock को नहीं रोकता — इसके लिए अलग techniques चाहिए होती हैं।

► Timestamp Ordering Protocol

हर transaction को शुरू होते ही एक unique timestamp मिलता है। Conflicting operations को उनके timestamp के order में execute किया जाता है, जिससे serializability सुनिश्चित होती है — locks की ज़रूरत नहीं पड़ती।

► Deadlock

🔗 Deadlock — जब दो या ज़्यादा transactions एक-दूसरे के resource का wait करती रहें, और कोई भी आगे ना बढ़ पाए ।

- Deadlock Prevention — पहले से ऐसी conditions avoid करना जिससे deadlock हो सके (जैसे Wait-Die, Wound-Wait schemes)
- Deadlock Detection — Wait-For Graph बनाकर cycle detect करना; cycle मिलने का मतलब deadlock है
- Deadlock Recovery — किसी एक transaction को rollback करके deadlock को तोड़ना

► Wait-Die और Wound-Wait Schemes

Scheme	Rule
Wait-Die	अगर requesting transaction older है, तो वो wait कर सकती है । अगर younger है, तो वो खुद को abort (die) कर देती है
Wound-Wait	अगर requesting transaction older है, तो वो younger transaction को abort (wound) कर देती है । अगर younger है, तो वो wait करती है

► Starvation

🔗 Starvation — जब एक transaction को बार-बार priority ना मिलने की वजह से हमेशा के लिए wait करना पड़े, भले ही deadlock ना हो । इसे रोकने के लिए FCFS (First-Come-First-Serve) जैसी fair policies इस्तेमाल होती हैं ।

► Multiple Granularity Locking

बड़े databases में हर बार पूरे table पर lock लेना inefficient होता है । इसलिए Multiple Granularity में locks को अलग-अलग levels पर लिया जा सकता है — Database → Table → Page → Row । इसके लिए Intention Locks (IS, IX) इस्तेमाल होते हैं, जो बताते हैं कि नीचे किसी level पर lock लिया जाने वाला है ।

► Optimistic Concurrency Control

🔗 Optimistic Concurrency Control — यह मान कर चलता है कि conflicts कम होते हैं, इसलिए locks का इस्तेमाल नहीं करता । इसके तीन phases होते हैं: Read Phase, Validation Phase, Write Phase — और सिर्फ commit के समय conflicts check किए जाते हैं ।

रिकवरी सिस्टम

Recovery System

🔑 Recovery — system crash, power failure या किसी और वजह से database corrupt होने पर उसे वापस एक consistent state में लाने की प्रक्रिया है ।

► Log-Based Recovery

हर transaction की हर operation को एक Log File में लिखा जाता है, इससे पहले कि वो actual database पर apply हो (Write-Ahead Logging - WAL) ।

Log Record	Explanation
<T, start>	Transaction T शुरू हुई
<T, X, old_value, new_value>	T ने data item X को update किया
<T, commit>	T successfully commit हुई
<T, abort>	T rollback हो गई

► Recovery Techniques

- UNDO — किसी uncommitted transaction के changes को वापस पुरानी state में लाना
- REDO — किसी committed transaction के changes को दोबारा apply करना (अगर वो disk पर save नहीं हो पाए थे)

► Checkpoints

🔑 Checkpoint — एक point है जहाँ system सारी memory की changes को disk पर permanently save कर देता है । Crash होने पर, recovery को पूरे log से नहीं बल्कि आखिरी checkpoint से शुरू करना पड़ता है — इससे recovery तेज़ होती है ।

► Shadow Paging

🔑 Shadow Paging — logging के बिना recovery की एक technique, जिसमें modify होने वाले pages की एक नई copy (shadow copy) बनाई जाती है, original page तब तक untouched रहता है जब तक transaction commit ना हो जाए ।

🔴 Shadow Paging में log की ज़रूरत नहीं पड़ती, पर यह fragmentation बढ़ा सकता है और concurrent transactions के साथ इस्तेमाल करना मुश्किल होता है ।

► Recovery Algorithms

Technique	Explanation
Deferred Update (NO-UNDO/REDO)	Transaction commit होने तक actual database में changes नहीं होते — सिर्फ log में लिखा जाता है । Crash पर सिर्फ REDO चाहिए
Immediate Update (UNDO/REDO)	Changes turant database में हो जाते हैं, log भी साथ-साथ maintain होता है । Crash पर UNDO और REDO दोनों चाहिए हो सकते हैं
ARIES Algorithm	एक popular modern recovery algorithm — तीन phases: Analysis, Redo, Undo

► Backup Types

- Full Backup — पूरे database की copy
- Incremental Backup — पिछले backup के बाद से हुए changes की copy
- Differential Backup — पिछले full backup के बाद के सभी changes

 Real systems में आम तौर पर हफ्ते में एक Full Backup + रोज़ाना Incremental Backup की strategy इस्तेमाल होती है — इससे storage भी बचता है और recovery भी तेज़ रहती है ।

इंडेक्सिंग और हैशिंग

Indexing & Hashing

📌 Index — एक data structure है जो किसी table में data को जल्दी ढूँढने में मदद करता है, ठीक वैसे जैसे किसी किताब का index पन्ने ढूँढने में मदद करता है ।

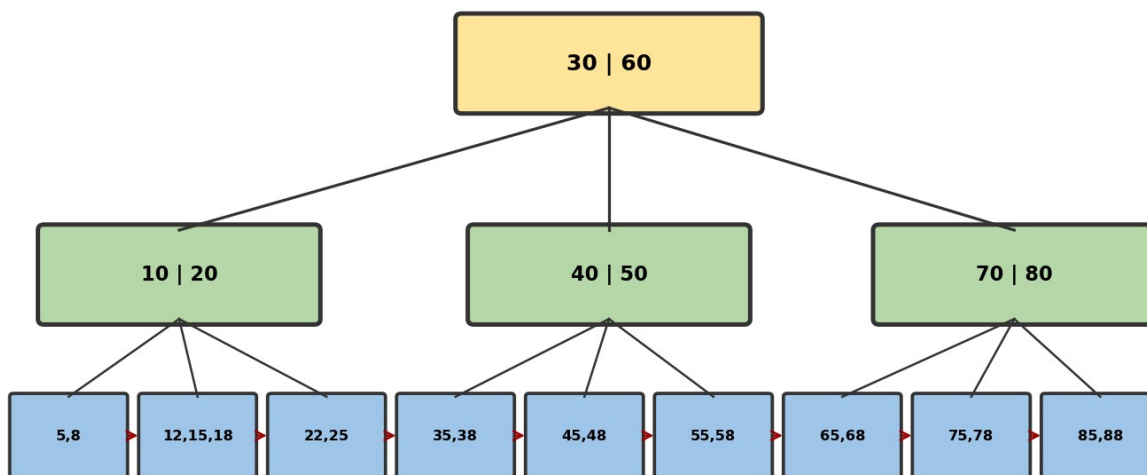
► Index के Types

Index Type	Explanation
Primary Index	Primary Key पर बना index, data physically sorted order में होता है
Clustering Index	एक non-key attribute पर बना index, जिसमें related records physically पास-पास रखे जाते हैं
Secondary Index	किसी non-primary attribute पर बना index, physical order से independent

► B-Tree और B+ Tree

ज़्यादातर databases indexing के लिए B+ Tree structure इस्तेमाल करते हैं क्योंकि इसमें search, insert और delete सभी operations efficient ($O(\log n)$) होते हैं ।

B+ Tree Index Structure (leaf nodes linked)



B+ Tree Index Structure

B-Tree	B+ Tree
Data, internal nodes में भी store होता है	Data सिर्फ leaf nodes में store होता है
Sequential access मुश्किल	Leaf nodes आपस में linked list की तरह जुड़े होते हैं — range queries आसान

► Hashing

🔑 Hashing — एक technique है जिसमें एक hash function, key value को सीधे उसके storage location (bucket) में map करती है, जिससे data access बहुत तेज़ होता है।

Type	Explanation
Static Hashing	Buckets की संख्या fixed रहती है
Dynamic Hashing	ज़रूरत के हिसाब से buckets की संख्या बढ़ती-घटती रहती है (जैसे Extendible Hashing)

🔥 Hashing exact-match queries (=) के लिए बहुत fast है, पर range queries (जैसे marks > 80) के लिए B+ Tree बेहतर रहता है।

► Multilevel Index

🔑 Multilevel Index — जब index खुद इतना बड़ा हो जाए कि memory में fit ना हो, तो उस index पर भी एक और index बनाया जाता है। B+ Tree इसी concept पर based है।

► SQL में Index बनाना

```
-- Simple index
CREATE INDEX idx_name ON Student(name);

-- Unique index
CREATE UNIQUE INDEX idx_email ON Student(email);

-- Composite (multi-column) index
CREATE INDEX idx_dept_name ON Student(dept_id, name);

-- Index delete karna
DROP INDEX idx_name;
```

🔥 Index हर column पर बनाना सही नहीं — इससे INSERT/UPDATE/DELETE धीमे हो जाते हैं क्योंकि हर बार index भी update करना पड़ता है। सिर्फ उन columns पर index बनाओ जिन पर बार-बार search/filter/join होता है।

फाइल ऑर्गनाइजेशन

File Organization

📌 File Organization — यह तय करता है कि records को disk पर physically कैसे arrange और store किया जाए।

Type	Explanation
Heap (Pile) File Organization	Records बिना किसी particular order के जहाँ जगह मिले वहाँ store होते हैं — insert तेज़, search धीमी
Sequential File Organization	Records किसी key attribute के according sorted order में store होते हैं — range search आसान
Hash File Organization	Hash function से record की location directly निकाली जाती है — exact search बहुत तेज़
Clustered File Organization	Related records (जैसे same department के employees) को physically पास-पास रखा जाता है

🔗 File Organization चुनते समय सोचो — ज्यादातर queries range-based हैं या exact-match? उसी हिसाब से Sequential या Hash चुनो।

► Record के Types

Type	Explanation
Fixed-Length Records	हर record का size same होता है — access करना आसान और तेज़
Variable-Length Records	अलग-अलग records का size अलग हो सकता है (जैसे VARCHAR fields की वजह से)

► Operations on Files

- Open — file को access के लिए तैयार करना
- Insert / Delete / Update — records के साथ काम करना
- Search (Retrieve) — किसी condition से records ढूँढना
- Reorganize — file को ज्यादा efficient structure में rearrange करना

एडवांस्ड टॉपिक्स

Advanced Topics

► NoSQL Databases

🔑 NoSQL — 'Not Only SQL' — ऐसे databases जो traditional relational (table-based) structure को follow नहीं करते, बड़े और unstructured data के लिए बनाए गए हैं।

NoSQL Type	उदाहरण	इस्तेमाल
Document-based	MongoDB	JSON जैसे documents store करना
Key-Value Store	Redis, DynamoDB	Simple key-value pairs, बहुत fast
Column-based	Cassandra, HBase	बड़ी मात्रा में columnar data
Graph-based	Neo4j	Relationships-heavy data (social networks)

► SQL vs NoSQL

SQL	NoSQL
Fixed Schema	Flexible / Dynamic Schema
Vertically Scalable	Horizontally Scalable
ACID Compliant	अक्सर BASE model follow करते हैं
Structured Data के लिए बेहतर	Unstructured / Semi-structured Data के लिए बेहतर

► Distributed Databases — बेसिक्स

🔑 Distributed Database — data कई अलग-अलग physical locations (servers) पर store होता है, पर users को यह एक single database जैसा दिखता है।

- Fragmentation — data को छोटे हिस्सों (horizontal/vertical) में बाँटना
- Replication — data की copies कई sites पर रखना (availability बढ़ाने के लिए)
- CAP Theorem — Consistency, Availability, Partition Tolerance — तीनों एक साथ पूरी तरह achieve नहीं हो सकते

► Database Security — बेसिक्स

- Authentication — user कौन है, यह verify करना (login/password)

- Authorization — user को क्या करने की permission है, यह control करना (GRANT/REVOKE)
- Encryption — sensitive data को unreadable form में store करना
- SQL Injection — एक common security threat जहाँ attacker malicious SQL code inject करता है — prepared statements से बचाव होता है

► Data Warehousing और Data Mining — बेसिक्स

- 📌 Data Warehouse — कई अलग-अलग sources से आया हुआ बड़ी मात्रा में historical data, जो analysis और reporting के लिए centrally store किया जाता है ।
- 📌 Data Mining — बड़े datasets में से patterns, trends और useful information निकालने की process (जैसे कौन से products साथ में बिकते हैं) ।
- 📌 OLTP vs OLAP — OLTP (Online Transaction Processing) रोजमर्रा के transactions के लिए है (जैसे बैंक transfer), जबकि OLAP (Online Analytical Processing) बड़े data पर analysis/reporting के लिए है ।

► Client-Server Architecture in DBMS

Architecture	Explanation
1-Tier	Client, Server और Database सब एक ही machine पर — testing के लिए इस्तेमाल
2-Tier	Client directly Database Server से बात करता है
3-Tier	Client → Application Server → Database Server — modern web apps में common (जैसे browser → backend API → database)

📌 3-Tier Architecture सबसे secure और scalable होती है क्योंकि client कभी database को direct access नहीं करता — यह हमेशा application server के through जाता है ।

क्वेरी प्रोसेसिंग और ऑप्टिमाइज़ेशन

Query Processing & Optimization

🔑 Query Processing — SQL query को submit करने से लेकर result मिलने तक की पूरी process है, जिसमें कई steps शामिल हैं।

► Query Processing के Steps

- Parsing और Translation — SQL query को check करके एक internal representation (relational algebra tree) में बदलना
- Optimization — query के कई possible execution plans में से सबसे efficient plan चुनना
- Evaluation — चुने गए plan को actually execute करना और result देना

► Query Optimization कैसे होता है

Query Optimizer अलग-अलग execution strategies (जैसे कौन सा join algorithm use करें, कौन सा index use करें) का cost estimate करता है, और सबसे कम cost (disk I/O, CPU time) वाला plan चुनता है।

Approach	Explanation
Heuristic (Rule-Based) Optimization	पहले से बने rules के आधार पर query को rewrite करना — जैसे SELECT को जल्दी apply करना ताकि rows कम हो जाएं
Cost-Based Optimization	हर possible plan का actual cost (statistics के आधार पर) estimate करके सबसे सस्ता plan चुनना

► Join Algorithms

Algorithm	Explanation
Nested Loop Join	हर outer row के लिए, पूरी inner table scan करना — simple पर धीमा
Sort-Merge Join	दोनों tables को join column पर sort करके फिर merge करना
Hash Join	छोटी table पर hash table बनाकर बड़ी table से match करना — बड़े datasets के लिए तेज़

🔑 Query Optimization Heuristic Rule: हमेशा SELECT (σ) और PROJECT (π) operations को जितना जल्दी हो सके apply करो, ताकि JOIN से पहले rows की संख्या कम हो जाए — इसे 'Selection Pushdown' कहते हैं।

► Query Execution Plan देखना (EXPLAIN)

```
-- MySQL/PostgreSQL mein query plan dekhna
EXPLAIN SELECT * FROM Employee WHERE dept_id = 5;

-- Detailed plan ke saath actual execution time
EXPLAIN ANALYZE SELECT * FROM Employee WHERE dept_id = 5;
```

 EXPLAIN command से पता चलता है कि query index इस्तेमाल कर रही है या पूरी table scan (full table scan) कर रही है — performance tuning के लिए यह सबसे पहला step है ।

डेटाबेस डिज़ाइन — केस स्टडी

Database Design — Case Study

अब तक जो भी पढ़ा, उसे इस्तेमाल करते हुए हम एक पूरा mini database design करेंगे — एक Library Management System के लिए।

► Requirements (ज़रूरतें)

- हर Book का एक ISBN, title, author और कितनी copies हैं, यह track होना चाहिए
- हर Member का नाम, membership_id, contact details रहें
- कोई Member किसी Book को Issue कर सकता है और वापस (Return) कर सकता है
- हर Issue की issue_date और due_date track होनी चाहिए

► Step 1: Entities और Attributes

Entity	Attributes
Book	isbn (key), title, author, total_copies
Member	membership_id (key), name, phone, email
Issue	issue_id (key), isbn (FK), membership_id (FK), issue_date, due_date, return_date

► Step 2: Relationships

Member और Book के बीच 'Issues' नाम का Many-to-Many relationship है (एक member कई books issue कर सकता है, एक book समय के साथ कई members को issue हो सकती है) — इसलिए एक junction table (Issue) की ज़रूरत है, जैसा हमने ER Model chapter में सीखा था।

► Step 3: SQL Schema बनाना

```
CREATE TABLE Book (
  isbn VARCHAR(20) PRIMARY KEY,
  title VARCHAR(100) NOT NULL,
  author VARCHAR(100),
  total_copies INT DEFAULT 1
);

CREATE TABLE Member (
  membership_id INT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  phone VARCHAR(15),
```

```

        email VARCHAR(100) UNIQUE
    );

CREATE TABLE Issue (
    issue_id INT PRIMARY KEY AUTO_INCREMENT,
    isbn VARCHAR(20),
    membership_id INT,
    issue_date DATE NOT NULL,
    due_date DATE NOT NULL,
    return_date DATE,
    FOREIGN KEY (isbn) REFERENCES Book(isbn),
    FOREIGN KEY (membership_id) REFERENCES Member(membership_id)
);

```

► Step 4: कुछ Useful Queries

```

-- Kaunsi books abhi tak return nahi hui
SELECT b.title, m.name, i.due_date
FROM Issue i
JOIN Book b ON i.isbn = b.isbn
JOIN Member m ON i.membership_id = m.membership_id
WHERE i.return_date IS NULL;

-- Har member ne kitni books issue ki hain
SELECT m.name, COUNT(*) AS books_issued
FROM Issue i
JOIN Member m ON i.membership_id = m.membership_id
GROUP BY m.name
ORDER BY books_issued DESC;

```

 यह exact pattern (Entity → Attributes → Relationships → Junction Table → SQL Schema) किसी भी real-world database design में इस्तेमाल होता है — इसे अच्छे से समझ लो तो कोई भी system design कर पाओगे।

► Codd's 12 Rules — RDBMS की पहचान (संक्षेप में)

E.F. Codd (जिन्होंने Relational Model invent किया) ने 12 rules बताए थे जो यह तय करते हैं कि कोई DBMS 'सच में' Relational है या नहीं। यहाँ कुछ मुख्य rules हैं:

- Information Rule — सारा data tables में ही represent होना चाहिए
- Guaranteed Access Rule — हर data value को table name, primary key और column name से access किया जा सके
- Systematic NULL Handling — missing information को systematically NULL से represent किया जाए

- Active Online Catalog — database का structure (metadata) भी उसी relational model से query हो सके
- View Updating Rule — जो views theoretically updatable हैं, वो system द्वारा भी updatable हों

एसक्यूएल प्रैक्टिस बैंक

SQL Practice Query Bank (30+ Queries)

नीचे दिए गए schema को base मानकर 30+ practice queries दी गई हैं — हर query के साथ छोटा सा explanation भी है, ताकि आप pattern समझ सकें।

```
-- Base Schema (is poore module ke liye)
Employee(emp_id, name, salary, dept_id, manager_id, join_date)
Department(dept_id, dept_name, location)
Project(project_id, project_name, dept_id)
WorksOn(emp_id, project_id, hours)
```

► Basic SELECT Queries

```
-- 1. Sabhi employees dikhao
SELECT * FROM Employee;

-- 2. Sirf naam aur salary chahiye
SELECT name, salary FROM Employee;

-- 3. 40000 se zyada salary wale employees
SELECT name FROM Employee WHERE salary > 40000;

-- 4. Naam ke hisaab se alphabetically sort karo
SELECT name FROM Employee ORDER BY name ASC;

-- 5. Sabse zyada salary wale 5 employees
SELECT name, salary FROM Employee ORDER BY salary DESC LIMIT 5;
```

► Filtering aur Pattern Matching

```
-- 6. Jin employees ka naam 'A' se shuru hota hai
SELECT name FROM Employee WHERE name LIKE 'A%';

-- 7. Salary 30000 aur 60000 ke beech
SELECT name FROM Employee WHERE salary BETWEEN 30000 AND 60000;

-- 8. Sirf dept_id 101, 102, 103 wale employees
SELECT name FROM Employee WHERE dept_id IN (101, 102, 103);

-- 9. Jinka manager assign nahi hai (NULL check)
```

```
SELECT name FROM Employee WHERE manager_id IS NULL;
```

► Aggregate aur GROUP BY

```
-- 10. Har department mein kitne employees hain
SELECT dept_id, COUNT(*) AS total FROM Employee GROUP BY dept_id;

-- 11. Har department ki total salary
SELECT dept_id, SUM(salary) AS total_salary FROM Employee GROUP BY dept_id;

-- 12. Sirf un departments ka average salary dikhao jinme 5+ employees hain
SELECT dept_id, AVG(salary) FROM Employee
GROUP BY dept_id HAVING COUNT(*) >= 5;

-- 13. Company ki highest aur lowest salary
SELECT MAX(salary) AS highest, MIN(salary) AS lowest FROM Employee;
```

► JOIN Queries

```
-- 14. Employee ka naam aur unke department ka naam
SELECT e.name, d.dept_name
FROM Employee e
JOIN Department d ON e.dept_id = d.dept_id;

-- 15. Un employees ki list jinka department nahi hai bhi (LEFT JOIN)
SELECT e.name, d.dept_name
FROM Employee e
LEFT JOIN Department d ON e.dept_id = d.dept_id;

-- 16. Kaunse employees kis project par kaam kar rahe hain
SELECT e.name, p.project_name, w.hours
FROM WorksOn w
JOIN Employee e ON w.emp_id = e.emp_id
JOIN Project p ON w.project_id = p.project_id;

-- 17. Self Join: har employee ka manager ka naam
SELECT e.name AS employee, m.name AS manager
FROM Employee e
LEFT JOIN Employee m ON e.manager_id = m.emp_id;
```

► Subqueries

```
-- 18. Average se zyada salary wale employees
SELECT name FROM Employee
WHERE salary > (SELECT AVG(salary) FROM Employee);

-- 19. 'IT' department mein kaam karne wale employees
SELECT name FROM Employee
WHERE dept_id = (SELECT dept_id FROM Department WHERE dept_name = 'IT');

-- 20. Un departments ki list jinka koi employee nahi hai
SELECT dept_name FROM Department
WHERE dept_id NOT IN (SELECT DISTINCT dept_id FROM Employee);
```

► Advanced Patterns

```
-- 21. Har department ka top earner (correlated subquery)
SELECT name, salary, dept_id FROM Employee e1
WHERE salary = (SELECT MAX(salary) FROM Employee e2 WHERE e1.dept_id = e2.dept_id);

-- 22. Duplicate naam wale employees dhoondo
SELECT name, COUNT(*) FROM Employee
GROUP BY name HAVING COUNT(*) > 1;

-- 23. CASE statement se salary band banao
SELECT name,
CASE
    WHEN salary < 30000 THEN 'Low'
    WHEN salary BETWEEN 30000 AND 60000 THEN 'Medium'
    ELSE 'High'
END AS salary_band
FROM Employee;

-- 24. Employee update karna (multi-table condition ke saath)
UPDATE Employee
SET salary = salary * 1.10
WHERE dept_id = (SELECT dept_id FROM Department WHERE dept_name = 'Sales');

-- 25. View banakar reuse karna
CREATE VIEW SeniorEmployees AS
SELECT name, salary FROM Employee WHERE join_date < '2018-01-01';

-- 26. Window function se rank nikalna (advanced)
SELECT name, salary,
    RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rnk
FROM Employee;
```

```
-- 27. Employees jo kisi project par kaam nahi kar rahe (NOT EXISTS)
SELECT name FROM Employee e
WHERE NOT EXISTS (SELECT 1 FROM WorksOn w WHERE w.emp_id = e.emp_id);

-- 28. Department-wise employee count, sirf non-zero
SELECT d.dept_name, COUNT(e.emp_id) AS cnt
FROM Department d
LEFT JOIN Employee e ON d.dept_id = e.dept_id
GROUP BY d.dept_name;

-- 29. Total hours kisi bhi employee ne sabhi projects par diye
SELECT e.name, SUM(w.hours) AS total_hours
FROM Employee e JOIN WorksOn w ON e.emp_id = w.emp_id
GROUP BY e.name
ORDER BY total_hours DESC;

-- 30. Second highest salary in each department (window function)
SELECT name, dept_id, salary FROM (
    SELECT name, dept_id, salary,
        DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rnk
    FROM Employee) t
WHERE rnk = 2;
```

 इन queries को खुद type करके किसी online SQL playground (जैसे SQLite Online, DB Fiddle) पर चलाकर देखो — hands-on practice से ही SQL पक्की होती है।

हल किए गए समस्याएँ

Solved Conceptual Problems

► समस्या 1 — Keys पहचानो

Table: Course(course_code, course_name, credits, dept_id). यहाँ हर course का code unique है, और course_name भी unique है (कोई दो courses का नाम same नहीं होता) ।

Question	Answer
Candidate Keys कौन-कौन सी हैं?	{course_code} और {course_name} — दोनों individually unique हैं
Primary Key कौन सी चुनोगे?	{course_code} — क्योंकि codes आमतौर पर names से ज़्यादा stable रहते हैं
Alternate Key कौन सी होगी?	{course_name}

► समस्या 2 — Normalization Identify करो

Table: StudentMarks(student_id, subject, marks, subject_teacher). यहाँ Primary Key = (student_id, subject), और subject_teacher सिर्फ subject पर depend करता है (student पर नहीं) ।

Question	Answer
क्या यह 1NF में है?	हाँ, सभी values atomic हैं
क्या यह 2NF में है?	नहीं — subject_teacher, पूरे composite key (student_id, subject) पर नहीं बल्कि सिर्फ subject पर depend करता है (partial dependency)
Solution क्या होगा?	दो tables बनाओ: StudentMarks(student_id, subject, marks) और SubjectTeacher(subject, subject_teacher)

► समस्या 3 — Transaction Schedule Analysis

दो transactions दी गई हैं: T1: Read(A), Write(A), Read(B), Write(B) और T2: Read(A), Write(A) । Schedule S: R1(A), R2(A), W1(A), W2(A), R1(B), W1(B) ।

Question	Answer
क्या यह schedule conflict serializable है?	चेक करने के लिए precedence graph बनाओ: W1(A) से पहले R2(A) है, और W2(A) से पहले W1(A) है, इसलिए T1→T2 और T2→T1 दोनों edges बन सकती हैं — अगर

Question	Answer
	cycle बनता है, तो यह Non-Serializable है
Practical lesson	हमेशा precedence graph draw करके cycle check करो, mental math से confuse ना हों

► समस्या 4 — ER to Relational Mapping

ER Diagram: Entity 'Employee' (emp_id, name), Entity 'Project' (proj_id, title), Relationship 'WorksOn' with attribute 'hours', cardinality M:N |

```
-- Solution: M:N relationship ke liye ek nayi junction table
Employee(emp_id, name)
Project(proj_id, title)
WorksOn(emp_id, proj_id, hours)  -- (emp_id, proj_id) is composite PK
    FOREIGN KEY(emp_id) REFERENCES Employee(emp_id)
    FOREIGN KEY(proj_id) REFERENCES Project(proj_id)
```

► समस्या 5 — Query Optimization Thinking

अगर एक query में WHERE salary > 50000 AND dept_id = 5 है, और dept_id पर index है पर salary पर नहीं, तो optimizer पहले dept_id वाला index इस्तेमाल करेगा (कम rows मिलेंगी), फिर बचे हुए rows पर salary filter apply करेगा — यह Selection Pushdown और index-based filtering का उदाहरण है ।

 Exam में conceptual problems solve करते समय हमेशा: (1) पहले definition/rule लिखो, (2) फिर step-by-step apply करो, (3) आखिर में final answer clearly बताओ — इससे partial marks भी मिलते हैं ।

एमसीक्यू प्रैक्टिस बैंक

MCQ Practice Bank

नीचे 24 Multiple Choice Questions दिए गए हैं जो beginner से advanced level तक cover करते हैं। पहले खुद answer सोचो, फिर नीचे दिया explanation पढ़ो।

► Q1-Q6: Basics

Question	सही Answer + Explanation
DBMS का full form क्या है?	Database Management System
File System के मुकाबले DBMS का सबसे बड़ा फ़ायदा क्या है?	Data Redundancy कम होना और Consistency बनी रहना
कौन सा DBMS component disk से data read/write करता है?	Storage Manager
Schema और Instance में क्या फर्क है?	Schema structure है (fixed), Instance actual data है (बदलता रहता है)
तीन levels में से कौन सा level users को सीधा दिखता है?	External Level
Logical Data Independence किस बात को refer करता है?	Conceptual schema बदलने पर External schema पर असर ना पड़ना

► Q7-Q12: Relational Model और Keys

Question	सही Answer + Explanation
Primary Key कभी क्या value नहीं ले सकती?	NULL
एक table में कितनी Foreign Keys हो सकती हैं?	एक से ज़्यादा — जितने चाहें
Super Key और Candidate Key में क्या फर्क है?	Candidate Key minimal होती है, Super Key में extra attributes हो सकते हैं
Referential Integrity Constraint किस पर लागू होता है?	Foreign Key पर — इसकी value parent table की existing key से match करनी चाहिए या NULL होनी चाहिए
Degree of a relation क्या दर्शाता है?	Table में columns (attributes) की संख्या
Cardinality of a relation क्या दर्शाता है?	Table में rows (tuples) की संख्या

► Q13-Q18: SQL

Question	सही Answer + Explanation
WHERE और HAVING में मुख्य फर्क क्या है?	WHERE rows को GROUP BY से पहले filter करता है, HAVING groups को बाद में
कौन सा command पूरी table का data delete करता है पर structure रखता है, और rollback नहीं हो सकता?	TRUNCATE
INNER JOIN क्या return करता है?	सिर्फ वो rows जो दोनों tables में match करती हैं
View क्या होता है?	एक query पर based virtual table, जिसमें actual data store नहीं होता
DISTINCT keyword किसलिए इस्तेमाल होता है?	Duplicate rows हटाने के लिए
GROUP BY के साथ किस तरह के functions इस्तेमाल होते हैं?	Aggregate functions जैसे COUNT, SUM, AVG

► Q19-Q24: Normalization, Transactions और Indexing

Question	सही Answer + Explanation
2NF सिर्फ किन tables पर लागू होता है?	जिनकी Primary Key composite (एक से ज़्यादा attributes की) हो
BCNF, 3NF से कैसे stronger है?	BCNF में हर determinant एक Super Key होना ज़रूरी है, 3NF में यह ज़रूरी नहीं
ACID में 'I' किसके लिए है और इसका मतलब क्या है?	Isolation — एक साथ चल रही transactions एक-दूसरे को affect नहीं करतीं
Deadlock detect करने के लिए किस structure का इस्तेमाल होता है?	Wait-For Graph — इसमें cycle मिलने पर deadlock होता है
B+ Tree में actual data कहाँ store होता है?	सिर्फ Leaf Nodes में
Hashing किस type की queries के लिए सबसे तेज़ है?	Exact-match (=) queries, range queries के लिए नहीं

 इन MCQs को 2-3 दिन बाद दोबारा बिना answer देखे solve करके देखो — spaced repetition से याद ज़्यादा पक्का होता है ।

► एक नज़र में — सारे Important Terms

Term	एक-लाइन Definition
DBMS	Database को manage करने वाला software
Primary Key	Row को uniquely पहचानने वाली key, NULL नहीं हो सकती
Foreign Key	दूसरी table से link बनाने वाली key
Normalization	Redundancy कम करने की process
ACID	Atomicity, Consistency, Isolation, Durability
Deadlock	Transactions का एक-दूसरे का wait करते रहना
Index	तेज़ search के लिए data structure
View	Query पर based virtual table
Schema	Database का logical structure
Join	दो या ज़्यादा tables को जोड़ना

► Practice Questions

- DBMS और File System में क्या अंतर है? Table बनाकर समझाओ ।
- ER Diagram बनाओ: एक Library System के लिए (Book, Member, Issue) ।
- Candidate Key, Primary Key और Foreign Key में अंतर बताओ ।
- 1NF, 2NF, 3NF और BCNF में क्या फर्क है? उदाहरण सहित समझाओ ।
- ACID properties को एक bank transaction के उदाहरण से समझाओ ।
- 2PL Protocol कैसे काम करता है? Growing और Shrinking Phase explain करो ।
- B+ Tree, B-Tree से कैसे अलग है?
- SQL में INNER JOIN और LEFT JOIN में क्या अंतर है? Query लिखकर दिखाओ ।
- Deadlock को कैसे detect और prevent किया जा सकता है?
- एक Student-Course Management System के लिए SQL schema (CREATE TABLE statements) लिखो, जिसमें Primary aur Foreign Keys हों ।

► Common Exam Mistakes से बचो

- WHERE और HAVING को मत मिलाओ — WHERE rows filter करता है, HAVING groups को
- DELETE और TRUNCATE दोनों DML/DDL अलग हैं — DELETE rollback हो सकता है, TRUNCATE नहीं
- 2NF सिर्फ composite primary key वाले tables पर apply होता है — यह भूल जाना common mistake है
- Candidate Key को हमेशा Primary Key समझने की गलती मत करो

 Best of luck! अगर कोई topic अभी भी unclear लगे, तो specific chapter दोबारा पूछ सकते हो — हम उसे और detail में explain कर सकते हैं !